

Day 1 — From Physical State to a Bit

Curriculum alignment: Stage 1 — Foundations of Computation · Electricity, States, Bits, and Information

Target time: ~90 minutes (≈45 min reading, ≈45 min observation and explanation)

Today's goal is not to memorize binary notation. It is to answer the more fundamental question: **how can a physical machine reliably represent anything at all?**

Why this day exists

Modern computers hide their ancestry well. You press a key, launch a process, or submit work to a GPU cluster; nowhere on the screen do you see the old problem that started the journey:

How can a physical object preserve a fact and transform it according to a repeatable rule?

Long before electronics, calculation was human labor. People used fingers, tallies, counting boards, and abaci because external physical state reduces the burden on memory. A moved bead remembers a partial result so the person does not have to.

Mechanical calculators pushed the same idea further. Pascal's seventeenth-century calculator used interlocking wheels to perform addition and subtraction. Leibniz pursued a stepped mechanism capable of multiplication and division. Babbage later designed machines in which storage, arithmetic, and control were increasingly distinct mechanisms.

These machines did not fail because their inventors lacked the idea of computation. Their central difficulty was implementation: precision gears are slow, wear out, occupy space, and become painfully complicated when thousands of states must interact.

Electronic computers are one answer to that physical-engineering problem. Before studying what a CPU does, we need to know what its circuits are manipulating. That is why the course begins with **state** rather than with C syntax or an instruction diagram.

The path we are starting is:

```
physical state
  → reliable symbols
  → logic and arithmetic
  → stored intermediate results
  → controlled sequence of operations
  → CPU
```

Today handles the first two arrows. Later lessons will earn the rest.

1. Start before binary: what is computation?

Suppose a system begins in one physical state and, according to some rule, changes into another:

```
initial state + rule + input → new state
```

That is the skeleton of computation.

- A mechanical calculator changes the positions of gears.
- An abacus changes the positions of beads.
- A relay computer changes which relays are open or closed.
- A modern processor changes electrical charge and voltage throughout a network of transistors.

The material is different, but the idea is the same: **computation requires physical state that can be changed and distinguished.**

Why must state be physical? Because information that affects the real world must exist in some real medium. A value cannot be stored “nowhere.” It may be represented by:

- voltage on a wire,
- charge in a capacitor,
- magnetic orientation,
- a transistor’s conductive state,
- pits on an optical disc,
- or the position of a mechanical object.

Software descriptions are abstractions over those physical states.

2. What is a state?

A **state** is the condition of a system at a particular moment.

Consider a light switch:

switch position: DOWN	switch position: UP
lamp state: OFF	lamp state: ON

The two switch positions are physically distinguishable. We can assign symbols to them:

```
DOWN → 0
UP   → 1
```

Nothing in nature says that DOWN *is* zero. We chose a mapping. We could swap the labels and the physical switch would behave identically.

This gives us our first crucial separation:

physical state		symbolic representation
DOWN / UP	↔	0 / 1

The state is physical. The symbol is an agreed description of that state.

Why does distinguishability matter?

Imagine a switch whose position randomly drifts and whose two intended positions are almost identical. You could no longer reliably answer whether it is “down” or “up.” The state would be poor storage.

A useful computing state must therefore be:

1. distinguishable from other states,
2. stable long enough to be used,
3. intentionally changeable,
4. readable without excessive ambiguity.

These requirements lead directly to digital electronics.

3. Electricity is not naturally binary

A common mental shortcut says:

```
0 volts = 0
5 volts = 1
```

This is useful at first, but physically inaccurate.

Voltage is continuous. A wire may be at 0.00 V, 0.21 V, 1.37 V, 3.29 V, or countless values in between. Real circuits also experience:

- thermal noise,
- crosstalk from nearby signals,
- voltage drops through resistance,
- power-supply fluctuations,
- manufacturing variation,
- and changing temperature.

Therefore a real digital input does not ask:

```
“Is the voltage exactly 0.000 V or exactly 3.300 V?”
```

It asks something like:

```
0.0 V ——— 0.8 V      2.0 V ——— 3.3 V
  valid LOW      unknown  valid HIGH
    → 0                → 1
```

The exact thresholds vary by logic family and device. The important idea is that **0 and 1 are ranges of acceptable physical values**, not perfect points.

The forbidden region

What should a circuit decide when the input is 1.4 V, between the guaranteed LOW and HIGH ranges?

The specification does not promise a stable logical result there. Different devices, temperatures, or moments could produce different answers. Designers try to ensure valid signals move through this region quickly rather than remaining there.

The gap between valid LOW and HIGH ranges creates a **noise margin**. Small physical disturbances can alter the voltage without changing the logical value.

```
ideal LOW signal:      0.2 V
noise adds:            +0.3 V
actual signal:         0.5 V
logical interpretation: still safely LOW
```

That tolerance is one of the central reasons digital systems can contain billions of switching elements and still behave predictably.

4. Why use only two logical states?

We could divide voltage into ten bands and directly represent decimal digits:

```
0.0–0.3 V → digit 0
0.3–0.6 V → digit 1
...
3.0–3.3 V → digit 9
```

This would put more information on one wire, but every band would be narrow. A small amount of noise could move a signal into the neighboring band. The receiver would also need to distinguish ten levels accurately.

With two states, we can use two wide regions separated by a large gap:

many states per wire	two states per wire
-----	-----
more information density	less information density
narrow noise margins	wide noise margin
complex receiver	simple switching circuit
harder to reproduce reliably	easier to restore reliably

Binary is therefore not the only imaginable system. It is a powerful engineering compromise:

Sacrifice information density per physical element in exchange for reliability, simple switching, and easy regeneration.

We recover capacity by combining many binary elements.

5. From two physical states to one bit

A **bit** is one binary digit: one of two distinguishable alternatives.

```
physical LOW → logical 0  
physical HIGH → logical 1
```

But be precise: the bit is not the voltage itself. Voltage is one possible physical implementation of a bit. Other technologies can represent the same logical alternatives differently.

IMPLEMENTATION		LOGICAL VALUE
low/high voltage	→	0 / 1
uncharged/charged capacitor	→	0 / 1
magnetic direction A/B	→	0 / 1
open/closed relay	→	0 / 1

This is an abstraction boundary:

```
messy physical world  
    ↓ thresholding and circuit design  
stable logical state: 0 or 1
```

Above that boundary, later layers can reason about exact bits without recalculating transistor physics for every operation.

6. Why adding a bit doubles the possibilities

One bit has two possible patterns:

```
0  
1
```

Add a second bit. For *each* old pattern, the new bit can independently be either 0 or 1:

```
00  
01  
10  
11
```

That gives four patterns. Add a third bit:

```
000 001 010 011 100 101 110 111
```

Eight patterns.

In general:

```
n bits → 2n distinct patterns
```

Each new independent bit doubles the previous set because every existing pattern now has two versions: one with the new bit equal to 0 and one with it equal to 1.

Bits	Distinct patterns	Unsigned values that can be encoded
1	2	0-1
2	4	0-3
3	8	0-7
4	16	0-15
8	256	0-255
16	65,536	0-65,535

Notice that “unsigned integer” is only one possible interpretation of those patterns. Day 2 will separate the pattern from its meaning.

7. Byte, word, and representation

Byte

A **byte** is eight bits grouped together:

```
10110100
```

Eight bits provide $2^8 = 256$ patterns. The eight-bit byte is a historical architectural convention, not a law of nature.

Word

A **word** is an architecture’s natural data width. In x86-64 discussions, “machine word” often means 64 bits, although x86 assembly also retains older names such as word (16 bits), doubleword (32 bits), and quadword (64 bits). Always establish the context rather than memorizing one universal size.

Representation

Bits are symbols for physical states; groups of bits are patterns. A representation rule tells us how to use those patterns.

For an unsigned four-bit integer, place values are powers of two:

```
bit position:    3   2   1   0
place value:     8   4   2   1
pattern:         1   0   1   1
value:           8 + 0 + 2 + 1 = 11
```

This is not because the pattern intrinsically contains “eleven.” It is because the unsigned-binary encoding assigns powers-of-two place values.

8. First observation on this Linux machine

We can ask the shell to show one value in several notations:

```
printf 'decimal: %d\nhex:      0x%X\noctal:  0%o\n' 70 70 70
```

Expected output:

```
decimal: 70
hex:     0x46
octal:   0106
```

The numeric value did not change. Only its written representation changed.

Now create one byte containing that value:

```
printf '\x46' > /tmp/day1-byte.bin
xxd -b /tmp/day1-byte.bin
xxd /tmp/day1-byte.bin
```

Expected views:

```
binary:      01000110
hexadecimal: 46
```

Predict before running

Before entering the next command, predict what it will print:

```
cat /tmp/day1-byte.bin
```

Why can a file containing the numeric byte value 70 appear as a letter on the terminal? Do not settle that question yet. Record your prediction. Day 2's lesson is built around answering it precisely.

9. Hands-on challenge

Do this without a calculator:

1. List every pattern possible with three bits.
2. Explain why there are eight without merely quoting 2^3 .
3. Convert 1101_2 to decimal using place values.
4. Convert decimal 19 to binary.
5. If LOW is guaranteed below 0.8 V and HIGH above 2.0 V: - classify 0.3 V, - classify 2.8 V, - explain why 1.4 V must not be trusted.
6. Run the Linux observation in §8 and record: - your prediction, - the actual result, - why the result raises a "meaning versus pattern" question.

Answers for self-checking after your attempt

1. 000, 001, 010, 011, 100, 101, 110, 111
2. Each of the four two-bit patterns has two extensions: one ending in 0 and one ending in 1.
3. $1101_2 = 8 + 4 + 0 + 1 = 13$
4. $19 = 16 + 2 + 1 = 10011_2$

5. 0.3 V is LOW; 2.8 V is HIGH; 1.4 V lies in the unspecified region.
6. `cat` displays `F`, introducing the fact that a reader supplies an interpretation to a stored pattern.

10. Mental model at the end of Day 1

```
physical quantity
  voltage/charge/magnetism
    ↓
engineered stable ranges
    ↓
two distinguishable logical states
    ↓
one bit: 0 or 1
    ↓
N bits create 2n patterns
    ↓
representation rules assign use to those patterns
```

Keep these distinctions:

- Electricity is continuous; digital logic is a designed abstraction.
- Logical 0 and 1 correspond to physical ranges, not exact voltages.
- Binary is practical because two wide state regions tolerate noise and need simple switching elements.
- A bit resolves one two-way choice.
- Adding one independent bit doubles the number of possible patterns.
- A pattern and its interpretation are different things.

11. Why? notebook

Write short answers or hypotheses:

1. If digital circuits use voltage ranges, what prevents a signal from staying forever in the forbidden region?
2. Why can adding more wires recover the information density lost by choosing only two voltage states?
3. Is a logical bit a physical object, an abstraction, or both depending on which layer we discuss?
4. Why is a byte eight bits rather than a law requiring some other size?
5. What exactly supplied the meaning “F” to the byte `01000110`?

12. Day 1 checkpoint

Without notes, explain this chain:

A real wire carries a continuous and noisy voltage. Circuit thresholds map broad voltage ranges into stable logical 0 and 1 states. One such two-way state is a bit. Combining n independent bits gives 2^n distinguishable patterns. Those patterns do not yet possess inherent meaning.

If any arrow in that explanation feels magical, record the question rather than memorizing past it.

Next: Day 2 — **From Voltage to Meaning:** quantify the noise-margin trade-off, define information and encoding precisely, and interpret one fixed bit pattern as a number, character, instruction, and pixel.

Historical references

- Charles Petzold, *Code*, Chapters 1–9, develops the path from human codes and switches to binary representation.
- IBM, “The punched card tabulator,” describes Hollerith’s later use of physical holes and electrical contacts to mechanize census data: <https://www.ibm.com/history/punched-card-tabulator>
- Computer History Museum, “Computers Timeline,” provides context for the later transition from mechanical to relay and electronic computation: <https://www.computerhistory.org/timeline/computers/>